



USER MANUAL

Learn parsing by practising

Interactive LL(1) and SLR(1) tutor

Version 2.0.0

Date 2026-10-04

GitHub github.com/jose-rZM/SyntaxTutor



Contents

1	Introduction	1
2	Installation	1
2.1	Windows	1
2.2	macOS	2
2.3	Linux	2
3	The home screen	2
3.1	Points and levels	2
3.2	The tutorial	3
4	Working with a tutor	4
5	LL(1) tutor	4
5.1	The LL(1) table	4
5.2	Guided mode	5
6	SLR(1) tutor	6
6.1	The LR(0) automaton	7
6.2	The SLR(1) table	7
6.3	Guided mode	8
7	Your own grammars	8
8	Exam mode	10
9	Exporting to PDF	10
10	Answer format	11
11	Settings and help	11
11.1	Language	11
11.2	Text size	11
11.3	Quick references	11
11.4	Where settings are stored	12
12	Developer documentation	12
13	License and credits	13

1 Introduction

SyntaxTutor is a desktop application for learning how **LL(1)** and **SLR(1)** parsers work. It works as a tutor that asks you questions step by step, checks every answer and, when you make a mistake, splits the problem into smaller questions until you see where it went wrong.

Every exercise starts from a grammar and walks through the whole algorithm: the FIRST and FOLLOW sets and the prediction symbols in LL(1); the initial item, closures, transitions and the collection of LR(0) states in SLR(1); and, at the end, the parsing table.

What is new in version 2.0

- **Your own grammars:** type the grammar you want to practise, with live validation (section 7).
- **Exam mode:** no help during the exercise and a grade from 0 to 10 at the end (section 8).
- **LR(0) automaton** drawn as you build the states (section 6.1).
- **Guided mode in LL(1) too**, to fill in the table one cell at a time.
- Redesigned interface, **adjustable text size** and renewed PDF exports.

This manual explains how to install the application and how to use each part. To get started, the **Tutorial** on the home screen covers the basics in a few minutes, and every question of the tutor states the answer format it expects.

2 Installation

SyntaxTutor needs no installer and no Internet connection. Download the latest version from the repository's releases page:

github.com/jose-rZM/SyntaxTutor/releases

- **Windows** 10 or 11, 64-bit:
SyntaxTutor-<version>-windows-x64.zip
- **macOS** on Apple Silicon (M1 or later):
SyntaxTutor-<version>-macos-arm64.zip
- **Linux x86_64** (Ubuntu 22.04, Debian 12 or newer):
SyntaxTutor-<version>-x86_64.AppImage

2.1 Windows

Unzip the archive and run SyntaxTutor.exe from the extracted folder; keep the other files next to the executable, as they are the libraries it needs.

2.2 macOS

Unzip the archive and move `SyntaxTutor.app` to *Applications*. The first time, open it with a right click and **Open**; if macOS still blocks it, go to *System Settings* > *Privacy & Security* and click **Open Anyway**.

2.3 Linux

The AppImage is a single executable file. Make it executable and run it:

```
chmod +x SyntaxTutor-<version>-x86_64.AppImage
./SyntaxTutor-<version>-x86_64.AppImage
```

It runs on X11 and, on Wayland desktops, through XWayland. To run it natively on Wayland, start it with `QT_QPA_PLATFORM=wayland`.

Security warnings

The Windows and macOS builds are not digitally signed, because signing requires a paid certificate. So Windows SmartScreen or macOS Gatekeeper may show a warning the first time. On Windows, click **More info** and then **Run anyway**. Only do so if you downloaded the file from the official repository.

3 The home screen

When SyntaxTutor opens, it shows the home screen (figure 1). From there you pick the tutor, the difficulty and how you want to practise.

- **LL(1)** and **SLR(1)** open the matching tutor.
- **Tutorial** starts a guided tour of the application (section 3.2).
- **Difficulty** sets the size of the exercise's random grammar: level 1 generates small grammars and level 3 the most complete ones.
- **Use my own grammar** opens the editor before starting, instead of generating a grammar (section 7).
- **Exam mode** removes the help and ends with a grade (section 8).
- **Language**, at the top right, switches between Spanish and English (section 11).

3.1 Points and levels

The top of the screen shows your points, your level and a progress bar. When you finish an exercise you gain one point for each correct answer and lose one for each wrong answer. Going from level n to $n + 1$ takes $10 \cdot n$ points, and there are 10 levels. If you leave an exercise halfway, it does not count. Progress is saved automatically.

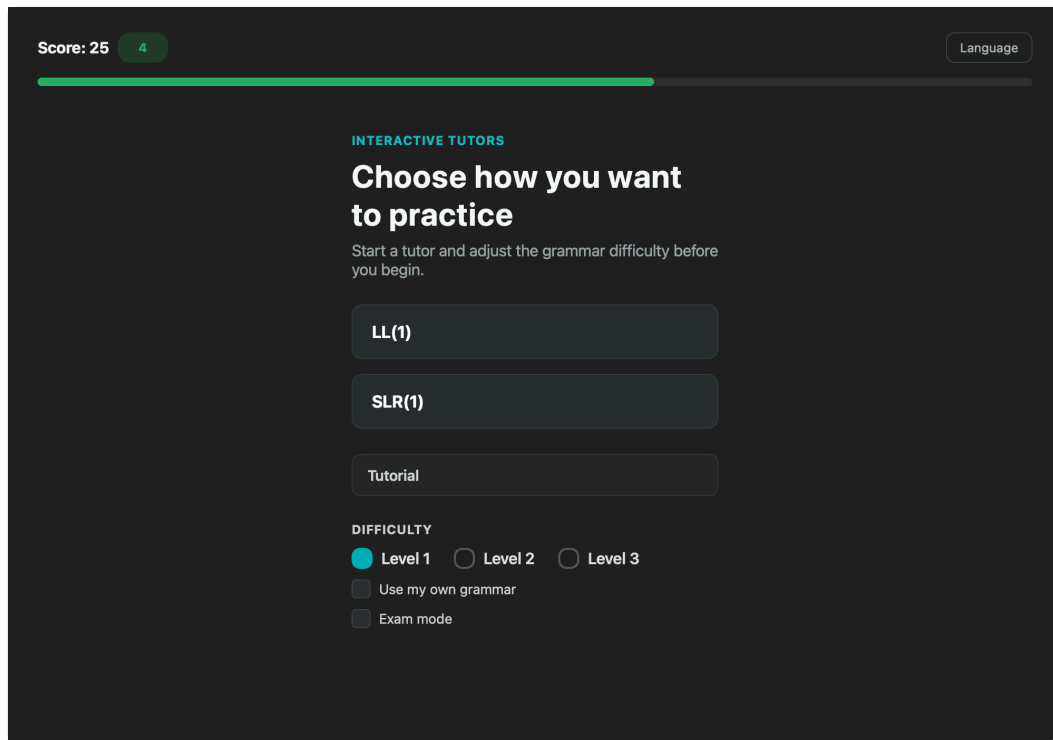


Figure 1 Home screen.

3.2 The tutorial

The **Tutorial** goes through the home screen, an LL(1) exercise and an SLR(1) exercise, highlighting each part of the interface (figure 2). Move on with **Next**. It is the quickest way to get started, and it is worth doing at least once.

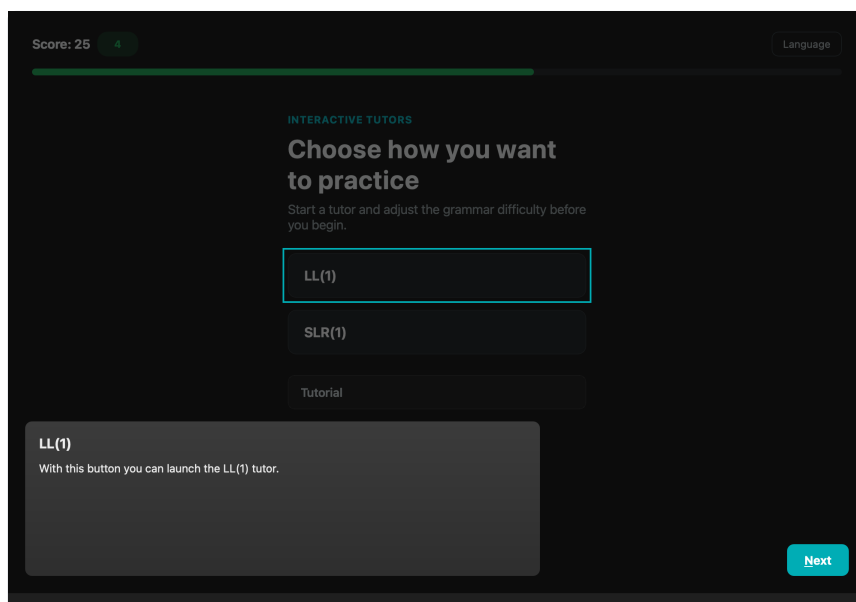


Figure 2 The tutorial highlights each element and explains what it is for.

4 Working with a tutor

Both tutors share the same window (figure 3):

- **Conversation**, on the left: the tutor asks the questions and you answer in the box at the bottom. Wrong answers are marked in red, and the counters at the top keep the score.
- **Grammar**, in the middle: the exercise's grammar, always visible.
- **Progress**, on the right: the results you have obtained so far (sets, states, transitions). Use it for reference, especially when filling in the final table.

Sending an answer. Type in the box at the bottom and press `Enter` or the send button. To write several lines (for example, a set of items in SLR(1)) use `Shift+Enter` or `Ctrl+Enter`; on macOS, `Cmd+Enter`. The grey text in the box reminds you of the expected format for each question.

When you make a mistake. The tutor explains what went wrong and, when needed, takes you through smaller questions that build the answer. Then it goes back to the original question.

When you finish. You can export the conversation to PDF (section 9) or exit. Points are applied when you leave with the final button. The **Back** button lets you leave the exercise at any time, but your progress is lost.

5 LL(1) tutor

The LL(1) exercise follows these steps:

1. **Table size.** How many rows and columns the LL(1) table has. Answer in the format rows,columns. If you get it wrong, the tutor asks separately for the number of non-terminals and terminals.
2. **Prediction symbols.** For each rule, its prediction symbols (PS). If you get it wrong, it first asks for the FIRST set of the right-hand side and, when needed, the FOLLOW set of the left-hand side, and then asks for the prediction symbols again.
3. **LL(1) table.** With all of the above, you fill in the predictive table.

5.1 The LL(1) table

The table is filled in its own dialog (figure 4). Each cell (A, a) holds the right-hand side of the rule for A that applies when reading a : write only the right-hand side symbols separated by spaces, or EPSILON for the empty rule. Leave cells without a rule blank. The tutor's progress panel stays available while you fill in the table.

Click **Finish** to submit it. On the first failed attempts, the wrong cells are marked in red. If you keep failing, the tutor gives reminders and, later on, offers you one last attempt.

Back 2 1

Which are the prediction symbols (PS) of this rule?
 $S \rightarrow A \$$
 Format: a,b,c
 22:25

User
 i,l
 22:25

Tutor
 Which are the prediction symbols (PS) of this rule?
 $A \rightarrow C B$
 Format: a,b,c
 22:25

User
 a, b
 22:25

Tutor
 You missed symbols.
 You have included symbols that do not belong: a, b.
 For a production $X \rightarrow Y$, its prediction symbols (PS) indicate which columns the production should be placed in the LL(1) table.
 The formula is: $PS(X \rightarrow Y) = FIRST(Y) - \{ \epsilon \} \cup FOLLOW(X)$ if $\epsilon \in FIRST(Y)$
 22:25

Tutor
 What is the FIRST set of the right-hand side?
 $A \rightarrow C B$
 Format: a,b,c
 22:25

Example: a,b,\$

Grammar rules:

$$\begin{aligned} S &\rightarrow A \$ \\ A &\rightarrow C B \\ B &\rightarrow p C B \\ &\quad | \text{ EPSILON} \\ C &\rightarrow E D \\ D &\rightarrow m E D \\ &\quad | \text{ EPSILON} \\ E &\rightarrow l A r \\ &\quad | i \end{aligned}$$

FIRST sets:
 • $FIRST(A \$) = \{l, i\}$

FOLLOW sets:
 • $PS(S \rightarrow A \$) = \{l, i\}$

Figure 3 LL(1) tutor. After a wrong answer, the tutor explains the mistake and asks for the FIRST set of the right-hand side.

	\$	i	l	m	p	r
S		A \$	A \$			
A		C B	C B			
B	EPSILON				p C B	EPSILON
C						
D						
E						

	\$	i	l	m	p	r
S		x	A \$			
A		C B	C B			
B	EPSILON				p C B	EPSILON
C		E D	E D			
D	EPSILON			m E D	EPSILON	EPSILON
E		i	l A r			

Figure 4 The LL(1) table half filled in and, on the right, after a submission with a wrong cell, which is marked in red.

5.2 Guided mode

If you get stuck, the **Guided mode** button goes through the table one cell at a time (figure 5). Each step shows the row and the column, gives a hint on how to reason about that cell and waits for your answer before moving on. Closing it takes you back to the table.

Row S, column 'i'

Step 1 of 15

Exit

HINT

Cell (S, 'i') is not empty: there is a production $S \rightarrow \alpha$ such that $'i' \in PS(S \rightarrow \alpha)$. Which production α do you write in this cell?

YOUR ANSWER

Write the production (EPSILON if empty)

Continue

Figure 5 Guided mode for the LL(1) table.

Back

8

0

22:24

Tutor

Calculate $\delta(I0, C)$: Which state do you get by transitioning with 'C'? This will be state number 2.

22:24

Tutor

Calculate $\delta(I0, i)$: Which state do you get by transitioning with 'i'? This will be state number 1.

22:24

Tutor

Calculate $\delta(I0, l)$: Which state do you get by transitioning with 'l'? This will be state number 3.

22:24

User

A $\rightarrow$ B.

B $\rightarrow$ B.mC

22:24

User

B $\rightarrow$ C.

22:24

User

C $\rightarrow$ i.

22:24

(0) S $\rightarrow$ A \$

(1) A $\rightarrow$ A p B

(2) | B

(3) B $\rightarrow$ B m C

(4) | C

(5) C $\rightarrow$ l A r

(6) | i

View LR(0) automaton

State I0:

[C $\rightarrow$. i]

[B $\rightarrow$. C]

[A $\rightarrow$. A p B]

[C $\rightarrow$. l A r]

[A $\rightarrow$. B]

[B $\rightarrow$. B m C]

[S $\rightarrow$. A \$]

Transitions:

o $\delta(I0, i) = I1$

o $\delta(I0, C) = I2$

o $\delta(I0, B) = I4$

o $\delta(I0, A) = I5$

State I1:

[C $\rightarrow$ i .]

State I2:

[B $\rightarrow$ C .]

State I4:

[A $\rightarrow$ B .]

[B $\rightarrow$ B . m C]

State I5:

[A $\rightarrow$ A . p B]

[S $\rightarrow$ A . \$]

Example: S $\rightarrow$. A \$ (Cmd + Enter for a new line)

Figure 6 SLR(1) tutor while the transitions of a state are computed. On the right, the numbered grammar and the states built so far.

6 SLR(1) tutor

The SLR(1) exercise has two parts: building the collection of LR(0) states and then the parsing table.

Initial state. First you are asked for the initial state, the closure of the axiom's item. If you get it wrong, the tutor asks step by step: which symbol comes after the dot, which rules have that

symbol as their left-hand side, and how the closure is completed.

LR(0) collection. Then the pending states are processed one by one. For each state you are asked how many items it has, which symbols come right after the dot and, for each symbol X , which state $\delta(I, X)$ is. If that state already exists, give its number; if it is new, it joins the queue of pending states.

State analysis. With the automaton complete, the tutor asks about the size of the table, the states with complete items, the states with LR(0) conflicts (in them, a reduction applies only on the FOLLOW symbols of its left-hand side) and the states that only reduce.

SLR(1) table. Finally, you fill in the parsing table.

6.1 The LR(0) automaton

The **View LR(0) automaton** button, above the progress panel, opens a window with the automaton (figure 7). The automaton is drawn as you build it: only the states and transitions you have already computed appear, and the state you are working on is highlighted. Use the wheel with **Ctrl** (**Cmd** on macOS) to zoom, drag to move around, and **Fit** to see the whole automaton.

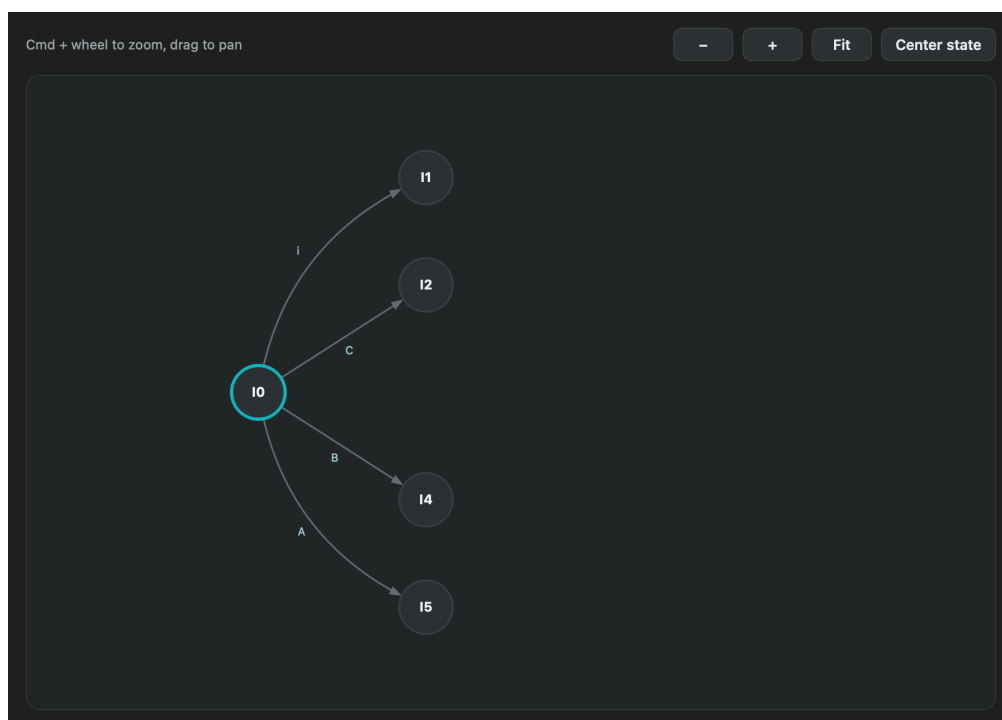


Figure 7 The LR(0) automaton after computing the transitions of the initial state.

6.2 The SLR(1) table

Rows are states, and columns are terminals (action) and non-terminals (goto). Each cell accepts:

Value	Meaning
s5	shift and go to state 5 (terminal columns)
r3	reduce by rule number 3
acc	accept
5	go to state 5 (non-terminal columns)
empty	error

Rules are numbered in the tutor's grammar; rule 0 is the augmented axiom's rule.

	i	l	m	p	r	\$	A	B	C	S
State 0	s1	s3					5	4	2	
State 1			r6	r6	r6	r6				
State 2			r4	r4	r4	r4				
State 3	s1	s3					6	4	2	
State 4			s7	r2	r2	r2				
State 5				s8		acc				
State 6										
State 7										
State 8										
State 9										
State 10										
State 11										

Guided mode
Finish

Figure 8 The SLR(1) table half filled in.

6.3 Guided mode

As in LL(1), **Guided mode** goes through the table state by state and symbol by symbol, with a hint at each step (figure 9).

7 Your own grammars

Tick **Use my own grammar** on the home screen and click **LL(1)** or **SLR(1)**. The editor opens (figure 10). It checks the grammar as you type and shows the axiom, the non-terminals, the terminals and the number of rules on the right.

Rules follow this format:

- Each rule has the form $A \rightarrow \text{symbols} \cdot$ and ends with a dot.
- Symbols are separated by spaces and can have several characters, such as `id` or `E'`.

State 0, symbol 'i' Exit

Step 1 of 45

HINT

State 0: there is a transition $\delta(0, 'i')$. To which state would you shift?

YOUR ANSWER

Enter your answer (e.g., s3, r2, acc, 5)

Continue

Figure 9 Guided mode for the SLR(1) table.

LL(1) EXERCISE

Write your grammar

Each rule ends with a dot and symbols are separated with spaces. The first left-hand side is the axiom, the left-hand sides are the non-terminals and every other symbol is a terminal. Use | for alternatives and leave the right-hand side empty for epsilon ($A \rightarrow \cdot$).

```
E -> T E' .
E' -> + T E' | .
T -> F T' .
T' -> * F T' | .
F -> ( E ) | id .
```

AXIOM
E

NON-TERMINALS
E E' F T T'

TERMINALS
() * + id

RULES
8

Valid LL(1) grammar. The initial rule $S \rightarrow E \$$ will be added.

Cancel Start exercise

Figure 10 The grammar editor with a valid LL(1) grammar.

- The first left-hand side is the axiom. Symbols that appear on a left-hand side are non-terminals; all others are terminals.
- | separates alternatives: $A \rightarrow a A \mid b \cdot$
- An empty right-hand side is the epsilon rule: $A \rightarrow \cdot$
- The characters `. , : ; | $` are reserved and cannot be part of a symbol.

Do not write the start rule with `$`: the editor adds $S \rightarrow \text{axiom } \$$ for you. The editor only lets you start the exercise when the grammar is LL(1) or SLR(1), depending on the tutor you chose. If it is not, the editor explains why. The last accepted grammar is remembered for next time.

8 Exam mode

With **Exam mode** ticked, the exercise becomes a test:

- There is no marking during the exercise: you are not told whether each answer is right, and the counters are hidden.
- Each question is answered only once. A mistake does not open smaller questions: the exercise moves on to the next one.
- Guided mode and the LR(0) automaton are not available.
- In the final table, each cell is graded separately.

At the end the report appears (figure 11): the grade from 0 to 10 (5 is a pass), the number of correct answers and the review of every question, with your answer and the correct one. **Export PDF** saves the report. Exam mode can be combined with your own grammars.

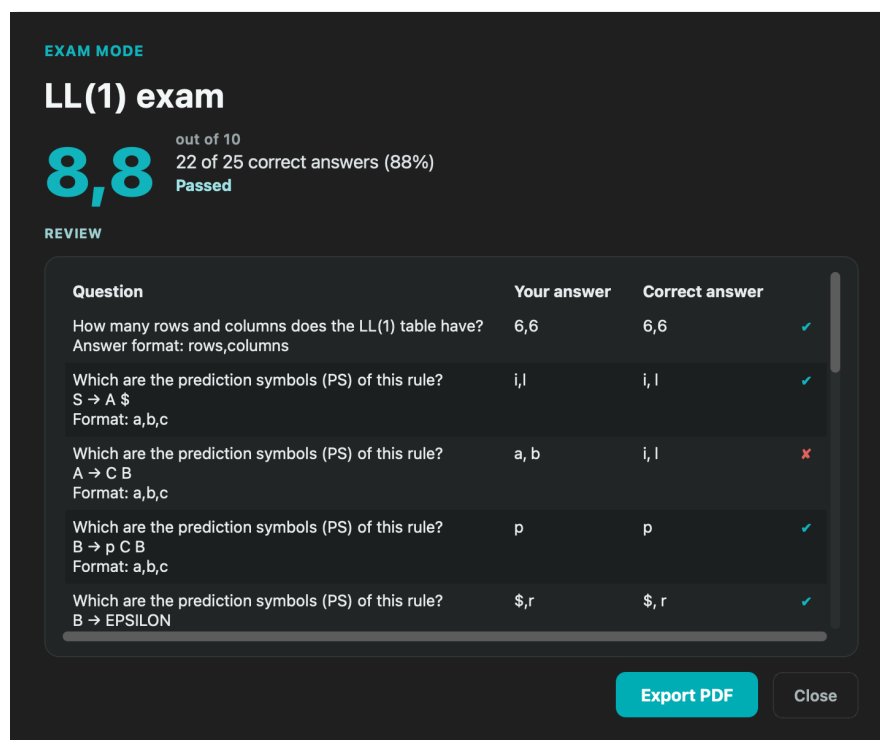


Figure 11 Report of an LL(1) exam.

9 Exporting to PDF

When you finish an exercise, **Export PDF** saves a document with:

- the grammar (numbered in SLR(1));
- the whole conversation, with the wrong answers highlighted;
- in LL(1), the FIRST and FOLLOW sets, the prediction symbols and the LL(1) table;

- in SLR(1), the automaton's states and the parsing table, in the same notation you use in the exercise.

Tables too wide for one page are split into blocks of columns, so large grammars stay readable too. The exam report is exported with the same design.

10 Answer format

Validation is strict, but every question states the format it expects. Spaces around commas and arrows do not matter.

Answer	Example
A number	4
Table size	5,7 (rows, columns)
Set of symbols	a, b, \$
Empty set	leave the answer blank
Rule	A -> a B
LR(0) item	A -> a . B (the dot, separated by spaces)
Set of items	one item per line (<code>Shift+Enter</code> for a new line)
LL(1) table cell	a B, or EPSILON
SLR(1) table cell	s3, r2, acc or 4

11 Settings and help

11.1 Language

Language, on the home screen, switches between Spanish and English (figure 12). The change applies when the application restarts.

11.2 Text size

The **Text size** menu offers several preset sizes and **Custom...** to pick one with a slider (figure 13). The size is saved.

11.3 Quick references

The **About** menu includes **LL(1) Quick Reference** and **SLR(1) Quick Reference**, two summaries of the theory you can look up at any time (figure 14). It also has **About SyntaxTutor**, with the version, the system and the credits.

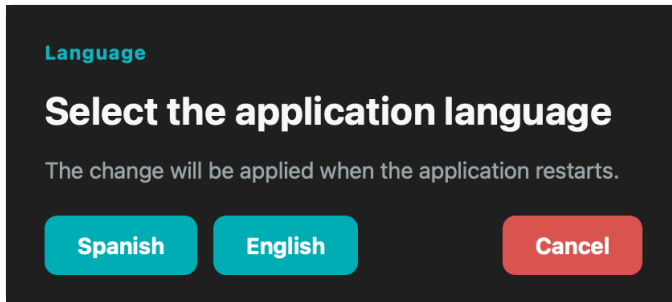


Figure 12 Language selection.

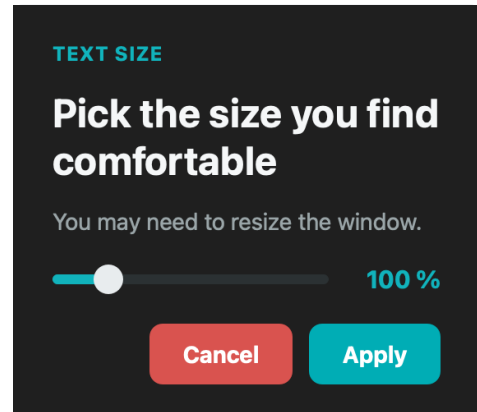


Figure 13 Text size.

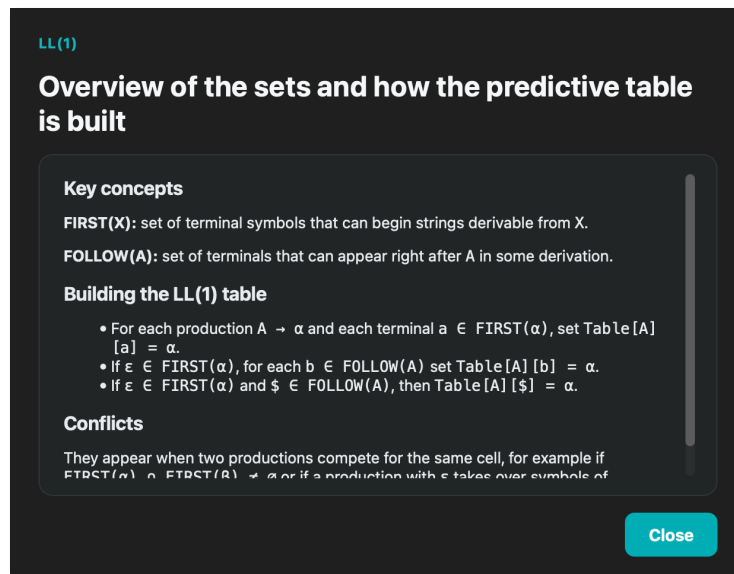


Figure 14 LL(1) quick reference.

11.4 Where settings are stored

Points, level, language, text size and the last grammar of your own are stored in:

- Windows: the registry, under HKEY_CURRENT_USER\Software\SyntaxTutor\SyntaxTutor.
- macOS: ~/Library/Preferences/me.jram.SyntaxTutor.plist.
- Linux: ~/.config/SyntaxTutor/SyntaxTutor.conf.

To start from scratch, delete that data while the application is closed.

12 Developer documentation

The source code is documented with Doxygen:

- Online: jose-rzm.github.io/SyntaxTutor.

- As a PDF: `manual/SyntaxTutor-Developer-Manual.pdf` in the repository.

The repository's README explains how to build the application and run the tests.

13 License and credits

SyntaxTutor is free software, released under the **GNU General Public License v3.0**. You may use it for any purpose, study how it works, change it and share it. Versions you distribute must keep the same license, keep the copyright notices and state the changes made. The application uses Qt 6, also under the GPLv3.

- **Author:** José R. (github.com/jose-rZM).
- **Origin:** SyntaxTutor version 1 was the Final Degree Project "Interactive Tutorial About Syntax Analyzers" (2025). Version 2 is a later extension and redesign, separate from that project.

If you use SyntaxTutor in class or build on it for another project, a mention of the author or the repository is appreciated.